# Employee Management Database

**Student Details:**

| | |
|---|---|
| Student Name | CHANDRU.MP |
| Register Number | 2428C0371 |
| Department | BSc AI&ML |
| ID | 98bc7bd2397c4bc8d9945b4f39d565b8@vtpt2026.in |
| College | AKSHAYA COLLEGE OF ARTS AND SCIENCE |
| Date of Submission | 25-9-2026 |
| GitHub Link | https://github.com/chandrump-2006/githubchan1.git |

## 1. Project Title

Student Database Management System

## 2. Problem Statement

Educational institutions manage large amounts of student information such as personal details, departments, courses, enrollments, marks, grades, and academic records. Managing this information manually or using separate files can lead to data duplication, errors, difficulty in searching records, and inefficient report generation.

The proposed system provides a centralized database to store, manage, update, search, and analyze student academic information efficiently.

## 4. Proposed Solution

Student360 is a centralized student database management system that connects student information with departments, courses, enrollments, and academic records. It provides CRUD operations, quick student search, course management, academic record management, and automated basic reports through a single system.

## 3. Objectives

- Maintain complete student information in a structured database.
- Manage departments and courses efficiently.
- Maintain student-course enrollment records.
- Store and manage marks, grades, and academic records.
- Provide quick student information search.
- Generate useful academic and administrative reports.

- Reduce manual paperwork and data duplication.

## 6. Main Modules

### Module 1 – Student Management

- Add new student records.
- View student information.
- Update student details.
- Delete inactive or incorrect records.
- Search students using ID, name, department, or year.

### Module 2 – Department Management

- Add and manage departments.
- Store department name and department ID.
- Connect students and courses with their respective departments.

### Module 3 – Course Management

- Add and update courses.
- Store course code, course name, credits, and department.
- View available courses.

### Module 4 – Student-Course Management

- Enroll students in courses.
- Maintain student-course relationships.
- Track courses taken by each student.
- View students enrolled in a particular course.

### Module 5 – Academic Record Management

- Store semester marks.
- Maintain grades and results.
- Calculate total marks and academic performance.
- Maintain semester-wise academic records.

### Module 6 – Search System

- Search student by Student ID.

- Search by student name.
- Search by department.
- Search by course.
- View complete student academic information.

## Module 7 – Report Generation

- Student-wise academic report.
- Department-wise student report.
- Course-wise enrollment report.
- Semester-wise result report.
- Basic performance report.

### 7. Database Tables Student

### Table

- Student_ID
- Student_Name
- Date_of_Birth
- Gender
- Email
- Phone
- Address
- Department_ID

### Department Table

- Department_ID
- Department_Name
- Department_Head

### Course Table

- Course_ID
- Course_Name
- Course_Code
- Credits
- Department_ID

### Student_Course Table

- Student_ID

- Course_ID
- Semester
- Enrollment_Date

### Academic_Record Table

- Record_ID
- Student_ID
- Course_ID
- Semester
- Marks
- Grade

## 8. Database Relationships

**Department → Students**
One department can have many students.

**Department → Courses**
One department can offer many courses.

**Student ↔ Course**
A student can take multiple courses and a course can have multiple students. This many-to-many relationship is managed through the **Student_Course** table.

**Student → Academic Records**
One student can have multiple academic records.

## 9. CRUD Operations

**Create:** Add students, departments, courses, enrollments, and academic records.

**Read:** Search and view student, course, department, and academic information.

**Update:** Modify student details, course information, enrollment details, and academic records.

**Delete:** Remove incorrect, duplicate, or inactive records.

## 10. Technology Stack

**Frontend:** HTML, CSS, JavaScript

**Backend:** Python / Flask

**Database:** MySQL

**Database Connectivity:** SQL

**Development Tool:** VS Code

**Reports:** SQL Queries / Dashboard

## 11. System Workflow

**Admin Login → Student Management → Department Management → Course Management → Student Enrollment →**
**Academic Record Entry → Search & Update → Report Generation**

## 12. Key Features

- **Centralized student database.**
- **Easy CRUD operations.**
- **Student-course relationship management.**
- **Academic record tracking.**
- **Fast search functionality.**
- **Automated basic reports.**
- **Reduced data duplication.**
- **Structured and scalable database design.**

## 13. Innovation / Enhancement

**The basic database can be extended into a smart academic management platform by adding:**

- **Attendance tracking.**
- **Automatic CGPA calculation.**
- **Student performance analytics.**
- **Low-performance alerts.**
- **Faculty login and role-based access.**
- **Interactive academic dashboard.**
- **Export reports as PDF or Excel.**
- **Future integration with AI-based student performance prediction.**

## 14. Expected Outcome

**The system provides an organized platform for storing and managing student information and academic records. It improves data accessibility, reduces manual work, supports accurate record management, and helps educational institutions generate useful reports quickly.**

## 15. Future Scope

**In the future, the system can be expanded into a complete college management platform by integrating attendance, fees, examination management, faculty management, placement records, notifications, mobile applications, and AI-based academic analytics.**

## 16. Database Design

| Table Name | Purpose |
|---|---|
| **Students** | **Stores student personal details** |

| | |
|---|---|
| **Departments** | **Stores department information** |
| **Courses** | **Stores available course details** |
| **Student_Course** | **Maintains student-course enrollment** |
| **Academic_Record** | **Stores marks, grades and semester results** |

## 17. Report Examples

| Report | Information Generated |
|---|---|
| **Department-wise Report** | **Number of students in each department** |
| **Course-wise Report** | **Number of students enrolled in each course** |
| **Student Academic Report** | **Marks and grades of a student** |
| **Semester Report** | **Semester-wise academic performance** |
| **Performance Report** | **Average marks of students** |

**18.Academic Records**

| Student ID | Student Name | Course | Semester | Marks | Grade |
|---|---|---|---|---|---|
| 1 | Arun Kumar | Python Programming | 3 | 88 | A |
| 1 | Arun Kumar | DBMS | 3 | 82 | A |
| 2 | Priya S | Machine Learning | 3 | 91 | A+ |
| 2 | Priya S | Artificial Intelligence | 3 | 86 | A |
| 3 | Karthik R | Python Programming | 3 | 76 | B+ |
| 4 | Divya M | Web Development | 3 | 89 | A |

## 19. Useful Database View

**A database view can be created to make student searches and academic reports easier.**

```
CREATE VIEW StudentDetails AS

SELECT

   s.student_id,

   CONCAT(s.first_name, ' ', s.last_name) AS student_name,

   s.email,

   s.phone,

   d.department_name,

   c.course_name,

   ar.semester,

   ar.marks,

   ar.grade

FROM Students s

JOIN Departments d

ON s.department_id = d.department_id

JOIN Academic_Record ar

ON s.student_id = ar.student_id

JOIN Courses c

ON ar.course_id = c.course_id;
```

## 20. Data Modification

**Modify Student Information**

```
UPDATE Students
```

**SET phone = '9888888888'**

**WHERE student_id = 1;**

**Modify Academic Record**

**UPDATE Academic_Record**

**SET marks = 90,**

**grade = 'A+'**

**WHERE student_id = 1**

**AND course_id = 1;**

**Remove Academic Record**

**DELETE FROM Academic_Record**

**WHERE student_id = 6;**

**Remove Student**

**DELETE FROM Students**

**WHERE student_id = 6;**

**The academic records and enrollment records should be handled first when foreign-key relationships depend on the student record.**

**21.Academic Performance Report**

**SELECT**

**s.student_id,**

**CONCAT(s.first_name, ' ', s.last_name) AS student_name,**

**AVG(ar.marks) AS average_marks**

**FROM Students s**

**JOIN Academic_Record ar**

**ON s.student_id = ar.student_id**

```
GROUP BY s.student_id, s.first_name, s.last_name;
```

**Find Students with Marks Greater Than 80**

```
SELECT
   s.student_id,
   CONCAT(s.first_name, ' ', s.last_name) AS student_name,
   ar.marks,
   ar.grade
FROM Students s
JOIN Academic_Record ar
ON s.student_id = ar.student_id
WHERE ar.marks > 80;
```

**22. Student Search**

**Find Student by Name**

```
SELECT *
FROM Students
WHERE first_name LIKE '%Arun%';
```

**Find Student by Email**

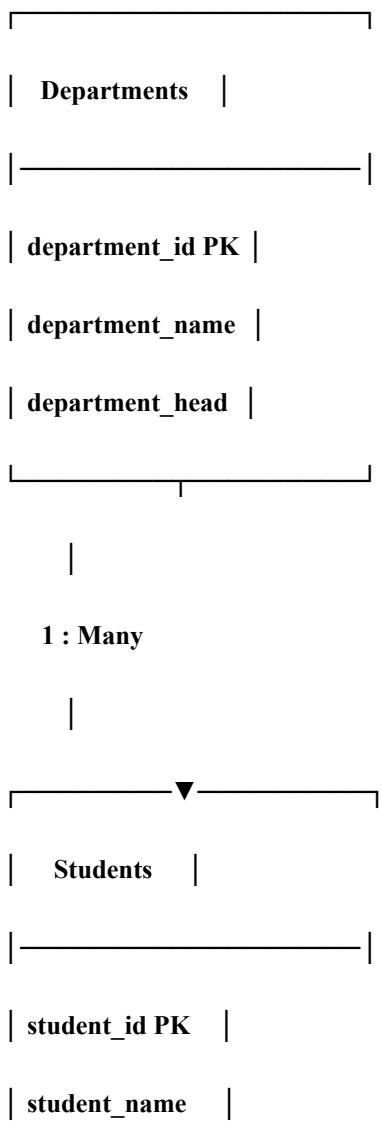
```
SELECT *
FROM Students
WHERE email = 'arun@gmail.com';
```

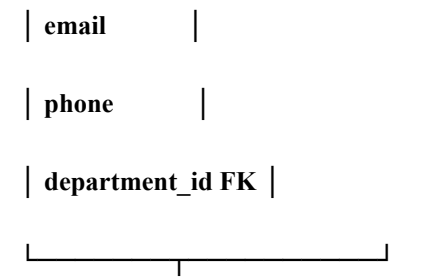
**Find Students by Department**

```
SELECT
   s.student_id,
```

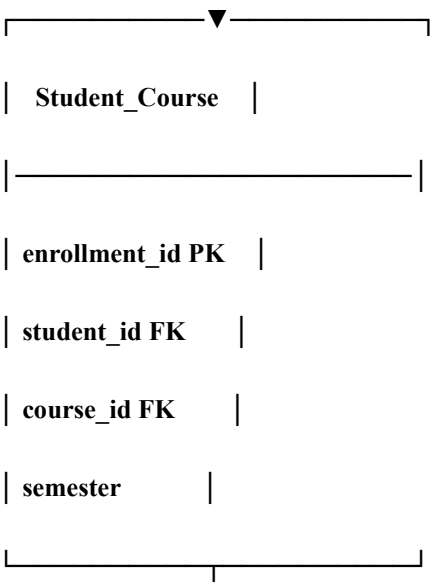
```
    s.first_name,
    s.last_name,
    d.department_name
FROM Students s
JOIN Departments d
ON s.department_id = d.department_id
WHERE d.department_name = 'Computer Science';
```

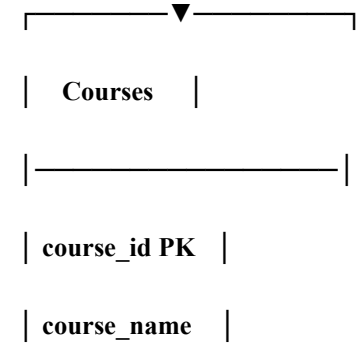
**Simple Relationship Diagram**

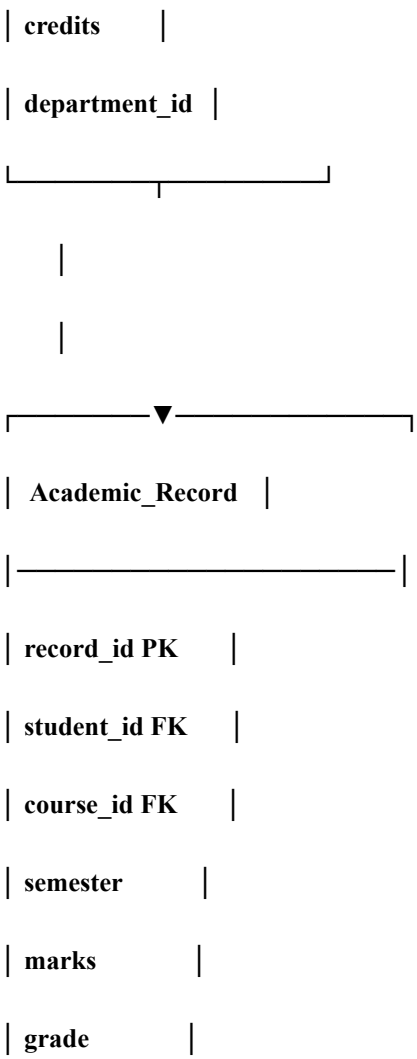



email
phone
department_id FK

Many : Many


Student_Course
enrollment_id PK
student_id FK
course_id FK
semester


Courses
course_id PK
course_name
course_code




**22.CONCLUSION**

**The proposed Student Information Management System is designed to provide a simple, efficient, and centralized solution for managing student-related information. The system maintains student personal details, departments, courses, course enrollments, and academic records in a structured database.**

**It reduces manual paperwork and minimizes errors in maintaining student records. The search and update features allow administrators and faculty members to access and manage information quickly. The reporting feature helps generate useful academic reports such as student performance, course details, and department-wise information.**

**The system can be further enhanced with attendance management, automated grade calculation, student performance analytics, notifications, role-based login, and a web-based dashboard. Overall, the project provides a reliable and scalable database solution that improves the efficiency of academic record management in educational institutions.**